

Synthetic Market Phase 21

Source-driven restoration across two axes on the compiled patch path: Leader (L4, top-4 components) and Dispersion (L35, top-4 components). Each axis uses 48 matched pairs, separately generated source behaviors, and a `swap_components` intervention that inserts source-side coefficients into the base prompt.

DATE	ROWS	PATCH	ENGINE
28 March 2026	48 per axis	swap_components, source-driven donor means	compiled non-eager custom-op

MAIN READ

Phase 21 is no longer a single-axis story. The Leader axis shows real partial restoration on tool-token choice and some positive movement on spend, while the Dispersion axis is weak or negative on tool selection and only clearly positive on spend for a one-row restorable subset. Both axes remain poor on response length.

LEADER RESTORE	TOOL-TOKEN	DISPERSION TOOL-TO- KEN RESTORE	LEADER RESTORATION	LENGTH	DISPERSION RESTORATION	LENGTH
0.438		0.167	-3.896		-16.512	
7 of 16 restorable rows		2 of 12 restorable rows	negative means the patched response moved away from the source length		much more negative than Leader	

What Was Tested

For each matched pair, the run produced three behaviors:

- the source prompt on its own
- the base prompt on its own
- the base prompt with a `swap_components` patch

The question is simple:

- did the patch make the base prompt behave more like its matched source prompt?

This report uses denoise pairs. In plain English, that means:

- the base row is the lower-valued member of the pair on the chosen metric
- the source row is the higher-valued member of the pair

Here the chosen metric depends on the axis:

- Leader uses `vol_1h_max`
- Dispersion uses `pct_1h_mad`

`swap_components` also has a plain-English meaning here:

- it does *not* copy every hidden-state token from source to base
- it captures the source row, averages over the market span, and extracts the selected coefficients
- it then writes those source-side coefficients into the base row over the base market span

One operational note matters for interpretation:

- Leader was run earlier at `batch_size = 8`
- Dispersion was run later at `batch_size = 32`
- both still used the same compiled non-eager custom-op patch path

Axis Comparison

Measure	Leader	Dispersion	How to read it
Source tool-name match	0.979 → 0.979	0.815 → 0.667	Dispersion gets worse overall; Leader was already saturated
Source tool-token match	0.66 → 0.723	0.556 → 0.407	Leader improves; Dispersion gets worse
Tool-token restoration rate	0.438	0.167	Higher is better on rows that needed fixing
Tool-token backfire rate	0.129	0.4	Lower is better on rows that were already correct
Spend improvement rate	0.571	1	Dispersion looks perfect, but only on one restorable row
Generated-token improvement rate	0.521	0.313	Both are weak; Leader is less bad
Mean normalized length restoration	-3.896	-16.512	Negative means the patch moved away from the source

Restorable Rows And Backfires

Metric	Leader	Dispersion
Tool name	restorable 1 restored 0 backfires 0 / 47	restorable 5 restored 1 backfires 5 / 22
Tool token	restorable 16 restored 7 backfires 4 / 31	restorable 12 restored 2 backfires 6 / 15
Spend pct	restorable 7 improved 4 backfires 5 / 47	restorable 1 improved 1 backfires 0 / 17
Generated tokens	restorable 48 improved 25 backfires 23 / 48	restorable 32 improved 10 backfires 25 / 48

This table is the clearest practical summary:

- Leader helps on tool-token choice more often than it hurts
- Dispersion hurts tool-token choice almost as often as it helps
- Dispersion spend looks good only because there was just one spend mismatch to fix
- neither axis gives a clean length-restoration story

Mechanical Validity

Measure	Leader	Dispersion
Rows with patch stats	48	48
Patch applied rate	1	1
Patch skipped rate	0	0

Mechanically, both runs are clean. Every row has patch diagnostics and nothing was skipped.

Methodology Scorecard

Best-practice item	Status
Matched prompt pairs and separately generated source behaviors	meets
Source-driven coefficient swap rather than generic ablation	meets
Compiled non-eager custom-op execution path for the patch itself	meets
Per-row patch diagnostics with zero skips on both axes	meets
Exact token-by-token source activation transplant	missing
Clean restoration of action choice on every tested axis	partial
Clean restoration of response length / verbosity	missing
Path-level validation	missing

The main methodological upgrade over Phase 20 is that Phase 21 uses *generated source behaviors* and a *source-driven swap*. That makes this a real restoration test, not only a disruption test.

Interpretation

Leader and Dispersion do not behave the same way.

Leader is the better causal candidate:

- it improves source match on the action surface we care about most
- tool-token restoration is materially above backfire
- spend moves toward the source more often than not

Dispersion is weaker:

- source match on tool name and tool token gets worse overall
- tool-token restoration is small and backfire is high
- the one clean positive is spend, but that result rests on a one-row restorable subset

The combined takeaway is not “restoration failed.” It is more specific:

- the market representation is not uniform
- Leader looks more like a usable causal handle on action choice
- Dispersion may matter for a narrower calibration-like quantity, but not for clean action restoration
- neither axis yet restores whole-response length cleanly

Next Step

The next experiment should push the restoration story where it is currently strongest:

- keep `batch_size = 32` as the default going forward unless a run errors
- extend the Leader restoration surface beyond `denoise` to other pairing directions
- keep Dispersion in the suite, but treat it as a contrast case rather than the primary success case
- if possible, move from span-mean coefficient swapping toward a more exact source-activation transplant

If Leader keeps restoring tool-token choice under those stronger tests, the causal claim becomes materially harder to dismiss.